Lab 8 Reflections

Antonio Garcia's Reflection

In this lab, I worked through the bias-variance tradeoff using polynomial regression models and learning curves. At first, the workflow felt like just a series of cells to complete, but as I went through each step, I started to see how the pieces connect into a full machine learning process. Generating the synthetic sine-wave data with noise helped me understand that real-world data is rarely clean, and that models have to balance fitting the pattern versus ignoring random fluctuations.

Training the models with degrees 1, 4, and 15 made the idea of underfitting and overfitting much more concrete. The degree 1 model was clearly too simple—it basically drew a straight line through curved data, which showed high bias. The degree 4 model felt like the "sweet spot" because it followed the overall trend without chasing every noisy point. The degree 15 model, on the other hand, was extremely wiggly and matched the training data almost perfectly, which looked impressive at first but obviously wouldn't generalize well.

Seeing all three curves on the same plot made the bias-variance tradeoff feel less abstract and more visual. The learning curves added another layer of understanding. For the underfitting model (degree 1), both the training and cross-validation scores were low and close together. That told me the model was consistently bad, no matter how much data it saw, which is a clear sign of high bias. For the overfitting model (degree 15), the training score was very high while the cross-validation score was much lower, creating a big gap between the two curves. That gap showed

high variance—the model was memorizing the training data but failing to generalize to new data. Instead of just reading definitions of bias and variance, I could actually see them in the plots.

Working through the notebook also helped me appreciate the importance of workflow structure. Using pipelines with PolynomialFeatures and LinearRegression made the code cleaner and showed how feature engineering and modeling can be chained together. The learning_curve function introduced the idea of evaluating models across different training set sizes, which is more informative than just looking at a single train/test split. I realized that understanding machine learning isn't just about knowing individual functions, but about seeing how data flows through each step and how each decision affects performance.

I did run into some technical issues, especially with cells getting compressed in Colab and functions not being recognized at first. Fixing those problems forced me to slow down and pay attention to details like function definitions, arguments, and execution order. That experience reinforced that debugging is part of the learning process, not just an annoyance. It also reminded me to question assumptions—for example, not assuming a function exists just because I see it on the page, but making sure the cell defining it actually ran.

Overall, this lab made the bias-variance tradeoff feel much more intuitive. Instead of thinking of it as a vague concept, I now see it as a balance between model complexity, training performance, and generalization. The visual examples and learning curves helped me connect the theory to actual behavior. Going forward, I'll be more conscious of checking both training and validation performance, and I'll think more carefully about whether a model is too simple, too complex, or appropriately matched to the data.

Khin Nyein Chan Thu

Lab 8 helped me understand why increasing model complexity does not always improve prediction quality. The lab uses synthetic data with a sine-wave pattern and random noise, making it possible to compare how different polynomial models represent the same information. This example clarified the difference between learning an underlying pattern and fitting every detail of the training data. A model needs enough flexibility to capture useful relationships, but too much flexibility can make it sensitive to noise.

The comparison of polynomial degrees 1, 4, and 15 helped explain underfitting and overfitting. A degree 1 model can only produce a straight line, which limits its ability to represent the curved pattern. This illustrates underfitting, where the model is too simple and has high bias. A degree 15 model has much more flexibility and can follow small changes in the data, including random fluctuations. This creates a risk of overfitting, where the model fits the training examples closely but may perform poorly on new data. The degree 4 model provides an intermediate level of complexity, although its usefulness still needs to be checked through evaluation.

The most meaningful idea for me was that a strong training score is not enough to prove that a model has learned well. A model may appear successful because it closely fits examples it has already seen. The real goal is generalization, which means making useful predictions on unfamiliar examples. This changes how I would choose a model: I would consider validation performance alongside training performance instead of assuming the most complicated model is the best choice.

Learning curves helped connect these ideas to the number of training examples. When both training and validation performance remain poor, the model may be underfitting. A large

gap between stronger training performance and weaker validation performance can suggest overfitting. These patterns help guide improvements. A model that underfits may need greater complexity or more useful features, while an overfit model may benefit from reduced complexity, regularization, or additional representative data.

This lab builds on Lab 7 because evaluation metrics and cross-validation help reveal problems with model complexity. It also shows that the machine learning workflow involves repeated decisions rather than simply training a model once. In cybersecurity, a model that memorizes familiar attack examples may struggle when attack behavior changes. My main takeaway is to use training and validation results together, investigate the reasons for poor performance, and choose a model that captures useful patterns while remaining reliable on new data.

Steven Ford

In this lab, working through polynomial regression models and learning curves helped bridge the gap between abstract machine learning theory and practical behavior. Generating synthetic sine-wave data with random noise immediately highlighted the challenge real-world data presents: models must capture underlying trends while remaining resilient to random fluctuations. Moving through each step of the notebook transformed what initially felt like executing isolated code cells into a cohesive process of understanding how data flows and how design choices impact overall performance.

Comparing models with polynomial degrees 1, 4, and 15 made the concepts of underfitting and overfitting concrete and visual. The degree 1 model produced a rigid straight line with high bias, failing to capture the curvature of the data. Conversely, the degree 15 model created an overly complex, wiggly curve that memorized the training points perfectly but sacrificed generalization. The intermediate degree 4 model struck the right balance, capturing the true pattern without chasing noise. Examining these curves side by side clarified why maximizing training accuracy alone is never a reliable indicator of success.

The introduction of learning curves added another dimension by illustrating how model performance changes across varying training set sizes. Analyzing the training and validation scores revealed distinct patterns: a narrow gap with poor scores indicated underfitting, while a large divergence with high training and low validation scores exposed high variance. Encountering and debugging minor technical hurdles along the way further reinforced the importance of careful execution and verification. Ultimately, this lab shifted my perspective on model evaluation, emphasizing the need to balance complexity and generalization rather than simply opting for the most intricate model available.

Cristian Delgado

Module 08 notebook explores underfitting and overfitting. It uses graphs to illustrate how a model reacts to certain parameters and how it affects the training and validity scores. Underfitting can be caused by several problems such as lack of training, insufficient data, poor features, and as a result generalizing too much. On the other hand, overfitting can be caused by excessive training, imbalanced data, and redundant features.

This causes a model to become too complex, and the model learns the data itself and is unable to generalize well on new data. In the notebook, after restructuring the code in the cells, I ran each cell and observed the results. The model representing underfitting was illustrated as a straight line and did not capture the nuance of the date points, which rose then fell and then rose and fell again. The model representing overfitting tried to capture too much nuance, resulting in a model that chased outlying data points and poor generalization. The good fit model featured a smooth curvy line that followed the data points without straying away to chase certain data points. In the score review, the underfitted model has a flat looking low score that represents a poor score that has peaked and will not improve regardless of how much extra data is added. The overfitted model training score is nearly 1 due to the fact that it essentially memorized the data set, and the y-axis is extremely skewed due to there being a large error in validation.

A question I have is if underfitting and overfitting is always obvious and clear cut as it was in these examples, or if the difference is sometime less clear and requires personal judgment when determining whether or not a model is finished training. In a similar vein, is there an acceptable amount of overfitting or underfitting, or is the goldilocks zone easily achieved enough times that these underperforming models are never acceptable?
